

Lab 1: Data Analysis with Python

Lab Outline:

- Installing the required packages
- Overview of Pandas, NumPy, Matplotlib and Seaborn

Installing Packages:

You can use **pip** to install packages. Check that you have a working Python with **pip** installed by running the following commands and making sure that the output looks similar:

```
C:> py --version
Python 3.N.N
C:> py -m pip --version
pip X.Y.Z from ... (python 3.N.N)
```

Then, use **pip** as follows:

```
pip install pandas
pip install scikit-learn
pip install matplotlib
pip install seaborn
```

Overview of Some Data Analysis Packages:

Pandas:

Pandas is an open source powerful Python library used for data analysis and manipulation.

Pandas provides two types of classes for handling data:

- **Series:** a one-dimensional labeled array holding data of any type such as integers, strings, Python objects etc.
- **DataFrame:** a two-dimensional data structure that holds data like a two-dimensional array or a table with rows and columns.

Import Pandas by writing:

```
import pandas
```

Let's create a series and a dataframe:

```
my_data = [5.1, 4.9, 4.7]
my_series =
pandas.Series(my_data)
print(my_series)
```

```
0      5.1
1      4.9
2      4.7
dtype: float64
```

```
my_data = {'Col1': [5.1, 4.9, 4.7], 'Col2': [3.5, 3.0, 3.2]}
my_df = pandas.DataFrame(my_data)
print(my_df)
```

```
   Col1 Col2
0  5.1  3.5  1
1  4.9  3.0
2  4.7  3.2
```

Dataset:

This dataset contains 2000 student records capturing essential academic, behavioral, and learning-related indicators used for analyzing patterns in student performance. It integrates key aspects of student activity, including attendance, study habits, engagement levels and assignment participation.

Each row represents an individual student with attributes reflecting learning behavior and academic involvement, along with a final performance outcome as the target variable.

Attendance	Prior_GPA	LMS_Login	Weekly_St	Assignmer	Engageme	Final_Academic_Performance		
91.27	2.45	194	26.6	96.7	43.33	High		
96.21	3.03	206	5.9	45.2	42.43	Medium		
96.83	1.16	119	23.1	84.67	68.21	High		
51.69	1.15	152	12.3	72.1	52.65	Medium		
60.51	3.1	99	1.6	41.7	48.59	Low		
44.72	3.98	278	37.6	38.89	76.61	High		
97.95	3.57	298	12.5	91.83	28.27	High		
52.15	3.77	183	34.8	43.64	81.43	Medium		

We will load the student data from the CSV file as follows:

```
studentsData = pd.read_csv("student_performance_dataset.csv")
type(studentsData)
pandas.core.frame.DataFrame
```

Explore the data:

```
print('Data Size:', len(studentsData)) # data.shape returns both dimensions
print('First Five Rows: \n', studentsData.head()) # default 5 rows
print('Last Three Rows: \n', studentsData.tail(3))
print(studentsData[120:122])
```

Data Size: 2000

First Five Rows:

	Attendance_Rate	Prior_GPA	LMS_Logins	Weekly_Study_Hours	\
0	91.27	2.45	194	26.6	
1	96.21	3.03	206	5.9	
2	96.83	1.16	119	23.1	
3	51.69	1.15	152	12.3	
4	60.51	3.10	99	1.6	

	Assignment_Submission_Rate	Engagement_Score	Final_Academic_Performance
0	96.70	43.33	High
1	45.20	42.43	Medium
2	84.67	68.21	High
3	72.10	52.65	Medium

...

To select certain data, we can use **loc** or **iloc**. The **loc** function requires row labels whereas the **iloc** function requires indexes (i.e. integer positions).

```
print(studentsData.iloc[0]) # prints the first row
print(studentsData.iloc[:, 0]) # prints the first column
print(studentsData.iloc[0, 0]) # prints the first cell
print(studentsData.iloc[20:101, 0:2])
print(studentsData.loc[:, 'Final_Academic_Performance'])
print(studentsData.loc[20:101, 'LMS_Logins']) # why does it work?
# print(studentsData.iloc[:, 'Final_Academic_Performance']) produces an error
# print(studentsData.loc[0,0]) produces an error
```

```
print(studentsData.values) # prints numpy representation, == data.to_numpy()
print(studentsData.T) # transpose
```

```
print(studentsData.columns)
print(studentsData.index)
```

```
Index(['Attendance_Rate', 'Prior_GPA', 'LMS_Logins', 'Weekly_Study_Hours',
      'Assignment_Submission_Rate', 'Engagement_Score',
      'Final_Academic_Performance'],
      dtype='object')
RangeIndex(start=0, stop=2000, step=1)
```

Manipulate the data:

```
studentsData.drop(columns=['Final_Academic_Performance'])
studentsData # the original data is not affected
```

	Attendance_Rate	Prior_GPA	LMS_Logins	Weekly_Study_Hours	Assignment_Submission_Rate	Engagement_Score	Final_Academic_Performance
0	91.27	2.45	194	26.6	96.70	43.33	High
1	96.21	3.03	206	5.9	45.20	42.43	Medium
2	96.83	1.16	119	23.1	84.67	68.21	High
3	51.69	1.15	152	12.3	72.10	52.65	Medium
4	60.51	3.10	99	1.6	41.70	48.59	Low
...	...	...	...	...	...	...	...

```
data2 = studentsData.copy()
data2.drop(columns=['Final_Academic_Performance'], inplace=True)
data2
```

	Attendance_Rate	Prior_GPA	LMS_Logins	Weekly_Study_Hours	Assignment_Submission_Rate	Engagement_Score
0	91.27	2.45	194	26.6	96.70	43.33
1	96.21	3.03	206	5.9	45.20	42.43
2	96.83	1.16	119	23.1	84.67	68.21
3	51.69	1.15	152	12.3	72.10	52.65
4	60.51	3.10	99	1.6	41.70	48.59
...	...	...	...	...	...	...

Note: We can use drop for dropping certain rows or columns by specifying the index/label of the row or column and the axis.

```
studentsData.drop(1, axis=0) # drops the second row
studentsData.drop('Final_Academic_Performance', axis=1) # drops the last column
```

```
studentsData['x0'] = [1]*2000 # inserts a column of 1s at the end
studentsData.insert(1, 'x1', [1]*2000)
studentsData.head()
#studentsData.drop(columns=['x0', 'x1'], inplace=True)
```

Attendance_Rate	x1	Prior_GPA	LMS_Logins	Weekly_Study_Hours	Assignment_Submission_Rate	Engagement_Score	Final_Academic_Performance	x0
91.27	1	2.45	194	26.6	96.70	43.33	High	1
96.21	1	3.03	206	5.9	45.20	42.43	Medium	1
96.83	1	1.16	119	23.1	84.67	68.21	High	1
51.69	1	1.15	152	12.3	72.10	52.65	Medium	1
60.51	1	3.10	99	1.6	41.70	48.59	Low	1

```
print(studentsData.sort_index(axis=1, ascending=True)) # axis = 0 to sort rows
print(studentsData.sort_values(by='Attendance_Rate'))
```

```
Assignment_Submission_Rate  Attendance_Rate  Engagement_Score \
0                          96.70           91.27           43.33
1                          45.20           96.21           42.43
2                          84.67           96.83           68.21
3                          72.10           51.69           52.65
4                          41.70           60.51           48.59
...
```

Calculate some statistics:

```
print(studentsData.mean())
print(studentsData.mean(1))
print(studentsData.median())
Attendance_Rate      69.784380
Prior_GPA             2.522000
LMS_Logins            151.328500
Weekly_Study_Hours    20.446050
Assignment_Submission_Rate 65.233195
Engagement_Score      50.487485
dtype: float64
0      75.725000
1      66.461667
2      65.495000
3      56.981667
4      42.416667
...
```

Note: you may work on a copy of data and drop the non-numeric column to avoid warnings.

```
print(studentsData.describe())
```

	Attendance_Rate	Prior_GPA	LMS_Logins	Weekly_Study_Hours	\
count	2000.000000	2000.000000	2000.000000	2000.000000	
mean	69.784380	2.522000	151.328500	20.44605	
std	17.503829	0.866343	83.727551	11.31047	
min	40.000000	1.000000	5.000000	1.00000	
25%	54.385000	1.780000	83.000000	10.50000	
50%	69.690000	2.540000	151.000000	20.50000	
75%	84.910000	3.270000	223.000000	30.20000	
max	99.980000	4.000000	299.000000	40.00000	

	Assignment_Submission_Rate	Engagement_Score
count	2000.000000	2000.000000
mean	65.233195	50.487485
std	20.437393	29.071301
min	30.000000	0.010000

```
print(studentsData.isna().sum())
print(studentsData['Final_Academic_Performance'].str.contains('Low').sum())
Attendance_Rate      0
Prior_GPA             0
LMS_Logins            0
Weekly_Study_Hours    0
Assignment_Submission_Rate 0
Engagement_Score      0
Final_Academic_Performance 0
dtype: int64
372
```

```
print(studentsData.groupby('Final_Academic_Performance').size())
print(studentsData['Final_Academic_Performance'].value_counts())
```

```
Final_Academic_Performance
High      705
Low       372
Medium    923
dtype: int64
Medium    923
High      705
Low       372
Name: Final_Academic_Performance, dtype: int64
```

Filter the data:

```
print(studentsData['Weekly_Study_Hours'] > 20)
print(studentsData[studentsData.get('Weekly_Study_Hours') > 20])
```

```
0      True
1     False
2      True
3     False
4     False
...
1995    True
1996   False
1997    True
1998   False
1999    True
Name: Weekly_Study_Hours, Length: 2000, dtype: bool
```

	Attendance_Rate	Prior_GPA	LMS_Logins	Weekly_Study_Hours \
0	91.27	2.45	194	26.6
2	96.83	1.16	119	23.1
5	44.72	3.98	278	37.6
7	52.15	3.77	183	34.8
8	68.48	3.41	115	21.1
...	...	...	...	...

NumPy:

NumPy is a fundamental package for scientific computing in Python. It is a library that provides a multidimensional array object and an assortment of routines for fast operations on arrays (including mathematical, logical, shape manipulation, discrete Fourier transforms, basic linear algebra, statistical operations, etc.).

Here is a sample of NumPy's functions and usage:

```
data1 = np.array(studentsData['Prior_GPA'])
data2 = studentsData['Attendance_Rate'].to_numpy()
print("Array 1 [Prior_GPA]:", data1)
print("Array 2 [Attendance_Rate]:", data2)
```

```
Array 1 [Prior_GPA]: [2.45 3.03 1.16 ... 3.52 1.07 3.3 ]
Array 2 [Attendance_Rate]: [91.27 96.21 96.83 ... 58.68 76.38 71.5 ]
```

```
tempArray = np.zeros(2000)
print(data1+ data2*(tempArray+2))
np.stack((data1,data2), axis = 1).shape
print(data1.shape)
newData = data1.reshape(1,-1) # argument -1: stands for Unknown Dimension
print(newData.shape)
np.multiply(np.ones((2000,1)), newData)

array([[2.45, 3.03, 1.16, ..., 3.52, 1.07, 3.3 ],
       [2.45, 3.03, 1.16, ..., 3.52, 1.07, 3.3 ],
       [2.45, 3.03, 1.16, ..., 3.52, 1.07, 3.3 ],
       ...,
studentsMatrix = np.array(studentsData.drop('Final_Academic_Performance',
axis=1))
studentsMatrix.shape

(2000, 6)

np.vstack((studentsMatrix, np.ones((1,4)))) # Stack arrays vertically (row-wise)

array([[9.127e+01, 2.450e+00, 1.940e+02, 2.660e+01, 9.670e+01, 4.333e+01],
       [9.621e+01, 3.030e+00, 2.060e+02, 5.900e+00, 4.520e+01, 4.243e+01],
       [9.683e+01, 1.160e+00, 1.190e+02, 2.310e+01, 8.467e+01, 6.821e+01],
       ...,
       [7.638e+01, 1.070e+00, 2.120e+02, 9.500e+00, 9.019e+01, 1.000e-02],
       [7.150e+01, 3.300e+00, 2.350e+02, 3.430e+01, 4.348e+01, 2.773e+01],
       [1.000e+00, 1.000e+00, 1.000e+00, 1.000e+00, 1.000e+00, 1.000e+00]])

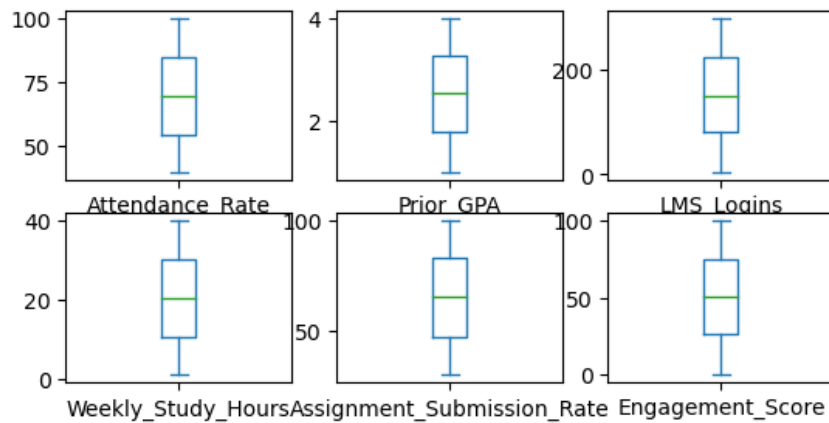
print(studentsMatrix.mean())
print(studentsMatrix.mean(0))
print(studentsMatrix.mean(1))

studentsMatrix.flatten()
array([ 91.27,   2.45, 194.  , ...,  34.3 ,  43.48,  27.73])
```

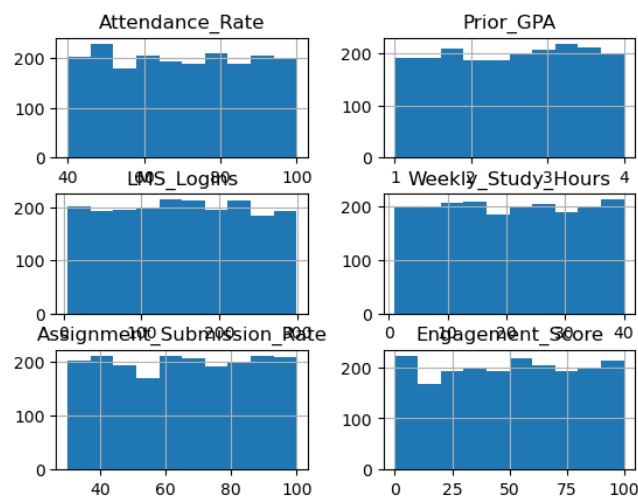
Matplotlib:

Matplotlib is a Python library that serves as a visualization utility. Here are some examples:

```
studentsData.plot(kind='box', subplots=True, layout=(3, 3), sharex=False,
sharey=False)
matplotlib.pyplot.show()
```



```
studentsData.hist()
matplotlib.pyplot.show()
```



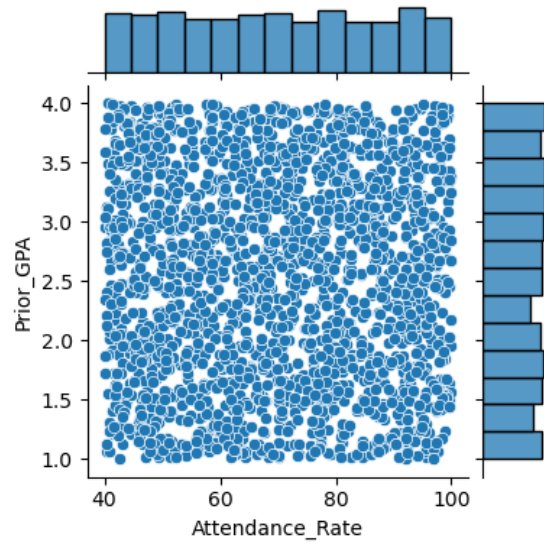
```
studentsData.plot(kind='scatter', x='LMS_Logins', y='Attendance_Rate')
matplotlib.pyplot.show()
pd.plotting.scatter_matrix(studentsData)
matplotlib.pyplot.show()
```

Seaborn:

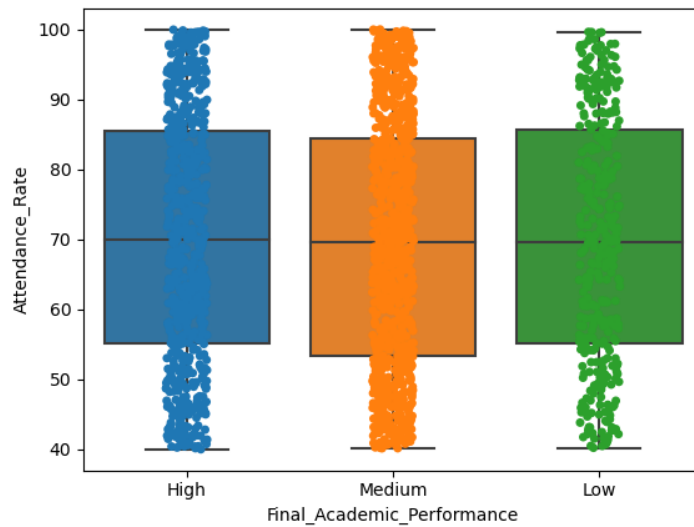
Seaborn is a Python data visualization library based on matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics.

Here is a sample of Seaborn's plots:

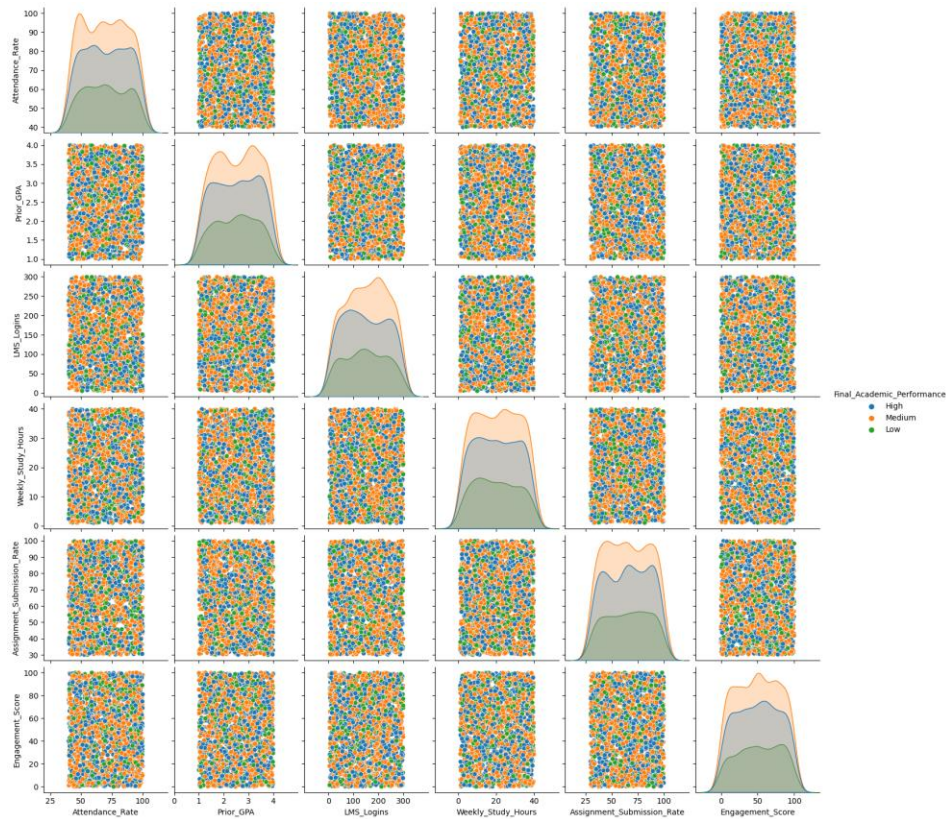
```
sns.jointplot(x='Attendance_Rate', y='Prior_GPA', data=studentsData, height=4)
```



```
combined = sns.boxplot(x='Final_Academic_Performance', y='Attendance_Rate',
data=studentsData)
combined = sns.stripplot(x='Final_Academic_Performance', y='Attendance_Rate',
data=studentsData,jitter=True, edgecolor='gray')
```



```
sns.pairplot(studentsData, hue='Final_Academic_Performance')
```



```
sns.heatmap(studentsData.corr(),annot=True,cmap='summer')
```

